

UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA
CORSO DI LAUREA MAGISTRALE IN INFORMATICA



TEST PLAN & REPORT

ANNO ACCADEMICO 2025/2026

Giuseppe Martusciello NF22500109
Giuseppe Sindoni NF22500108

Contents

1	Introduzione	4
1.1	Obiettivi della sezione di Testing	4
1.1.1	Regression Testing	4
1.2	Evoluzione storica della Test Suite	5
1.2.1	Stato iniziale del progetto	5
1.2.2	Introduzione della baseline di test	6
1.2.3	Crescita incrementale guidata dalle Change Request	6
1.2.4	Integrazione con CI/CD e Sonar	6
2	Architettura e Stato della Test Suite	8
2.1	Framework e Strumenti di Testing	8
2.2	Struttura della Test Suite	8
2.3	Uso del Mocking	9
2.4	Metriche di Coverage	9
2.5	Integrazione con CI/CD e Analisi Statica	10
3	Strategia di Testing e Regression Testing	11
3.1	Processo di testing dopo una Change Request	11
3.1.1	Identificazione delle aree impattate	11
3.1.2	Moduli e layer coinvolti	11
3.2	Strategia di regression testing	11
4	Criteri di Adeguatezza del Testing	13
4.1	Criteri White-Box (Testing White Box)	13
4.1.1	Coverage minimo dichiarato	13
4.1.2	Dove è più rilevante la Branch Coverage	14
4.2	Criteri Black-Box e Integration Testing	15
4.2.1	Copertura funzionale tramite test di integrazione	16
4.2.2	Strategia di mocking	16
4.2.3	Esempio di flusso testato	16

4.2.4	Esecuzione dei test	17
5	Infrastruttura di Test e Qualità del Progetto	18
5.1	Test Runner e TypeScript	18
5.1.1	Test Runner	18
5.1.2	Configurazione TypeScript	18
5.1.3	Esecuzione dei Test	18
5.2	Coverage Tool (Istanbul/Jest)	19
5.2.1	Motore di Coverage	19
5.2.2	Specifiche di Coverage	19
5.2.3	Generazione del Report	19
5.3	Continuous Integration (CI)	19
5.3.1	Piattaforma di Integrazione Continua	19
5.3.2	Esecuzione dei Test in CI	20
5.4	Analisi Statica (SonarCloud/SonarQube)	20
5.4.1	Piattaforma di Analisi Statica	20
5.4.2	Configurazione di SonarCloud	20
5.4.3	Metriche di Qualità Monitorate	20
5.5	Gestione Dati di Test	21
5.5.1	Mock e Fixtures	21
5.5.2	Database	21
5.5.3	In-memory DB	21
5.6	Riproducibilità dei Test	21
5.6.1	Locale vs CI	21
5.6.2	Variabili d'Ambiente	22
6	Assessment e Miglioramenti Futuri	23
6.1	Risultati ottenuti	23
6.2	Limiti attuali e possibili miglioramenti	23
7	Change Request #1	25
7.1	Stato del sistema di testing prima della Change Request	25
7.2	Test introdotti per la Change Request	26
7.3	Strategia di regression testing	27
7.4	Risultati dell'esecuzione dei test	27
7.5	Coverage e miglioramento della qualità	28

7.6	Valutazione finale del testing della CR	28
8	Change Request #2	29
8.1	Stato del sistema di testing prima della Change Request	29
8.2	Test introdotti per la Change Request	29
8.3	Strategia di regression testing	31
8.4	Risultati dell'esecuzione dei test	31
8.5	Coverage e miglioramento della qualità	31
8.6	Valutazione finale del testing della CR	32
9	Change Request #3	33
9.1	Stato del sistema di testing prima della Change Request	33
9.2	Test introdotti per la Change Request	33
9.3	Strategia di regression testing	35
9.4	Risultati dell'esecuzione dei test	36
9.5	Coverage e miglioramento della qualità	36
9.6	Valutazione finale del testing della CR	37

1 Introduzione

Questo documento descrive la strategia di testing adottata nel progetto **ISTA2026**.

Le attività di testing sono state introdotte e sviluppate progressivamente durante le attività di manutenzione evolutiva del sistema. In particolare, la test suite è stata costruita e ampliata nel tempo per supportare l'introduzione delle Change Request (CR) e per garantire la stabilità del sistema attraverso attività di regression testing e analisi automatica integrate nella pipeline CI/CD.

1.1 Obiettivi della sezione di Testing

Questa sezione descrive l'approccio adottato per il testing del sistema, le tecniche utilizzate e gli strumenti impiegati per verificare il corretto funzionamento dell'applicazione durante il processo di evoluzione del software.

In particolare vengono presentati:

- il ruolo delle attività di testing all'interno del progetto
- l'integrazione tra testing e manutenzione evolutiva
- il processo di introduzione e crescita della test suite
- l'utilizzo di strumenti automatici di analisi e integrazione continua

L'obiettivo principale della strategia di testing è garantire che le modifiche introdotte nel sistema mantengano il corretto funzionamento delle funzionalità esistenti e non introducano regressioni nel comportamento dell'applicazione.

1.1.1 Regression Testing

Nel progetto le attività di regression testing sono utilizzate per verificare che le modifiche introdotte nel sistema non compromettano il funzionamento delle funzionalità già presenti.

A tale scopo è stata implementata una suite di test automatizzati che viene eseguita ogni volta che il codice viene modificato o aggiornato. L'esecuzione completa della test suite consente di individuare rapidamente eventuali regressioni introdotte durante lo sviluppo o l'integrazione delle nuove funzionalità.

1.2 Evoluzione storica della Test Suite

La test suite non era presente nelle prime fasi di sviluppo del sistema ed è stata introdotta successivamente durante le attività di manutenzione evolutiva del progetto.

L'introduzione del testing automatizzato è stata effettuata in modo incrementale, con l'obiettivo di migliorare progressivamente l'affidabilità del sistema e supportare l'evoluzione del codice nel tempo.

1.2.1 Stato iniziale del progetto

Nella fase iniziale del progetto il sistema non disponeva di una suite di test automatizzati. La verifica del corretto funzionamento delle funzionalità veniva eseguita manualmente durante lo sviluppo.

Questa situazione comportava alcune limitazioni operative, tra cui:

- maggiore difficoltà nel verificare rapidamente il comportamento del sistema
- aumento del rischio di introdurre errori durante le modifiche
- assenza di metriche automatiche relative alla qualità del codice

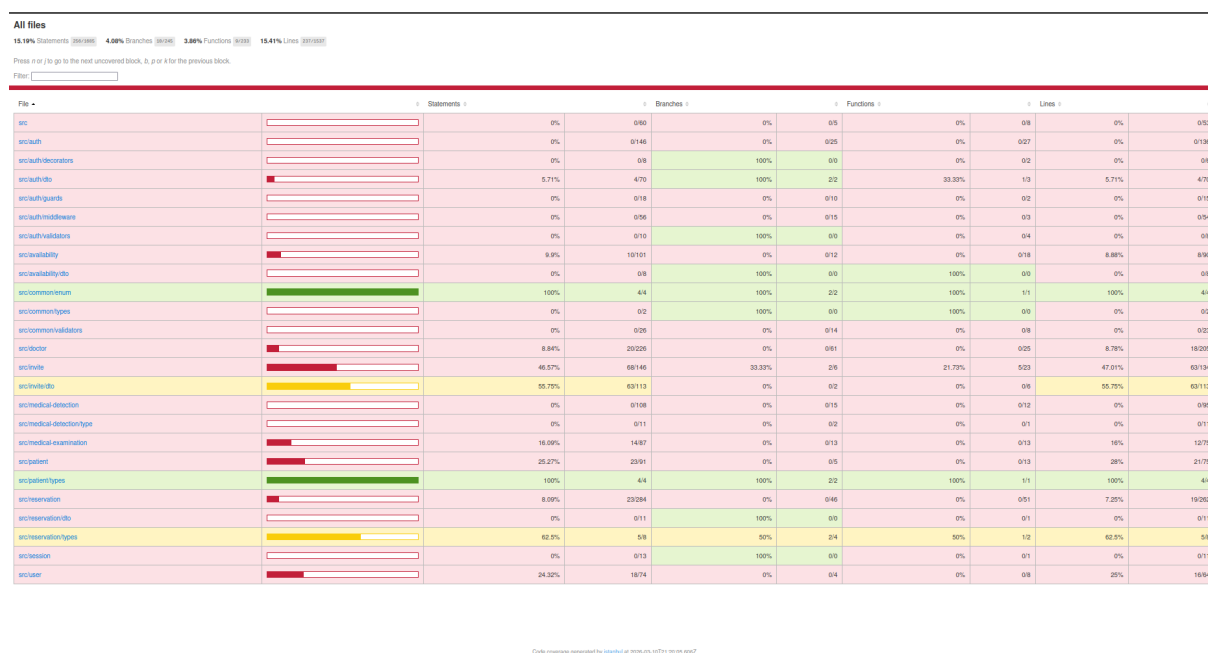


Figure 1.1: Situazione iniziale suite di test

Per migliorare l'affidabilità del sistema è stata quindi introdotta una strategia di testing strutturata.

1.2.2 Introduzione della baseline di test

La prima fase di miglioramento dell'infrastruttura di testing ha previsto l'introduzione di una **baseline iniziale di test**.

In questa fase è stata implementata una prima suite di test unitari finalizzata a coprire le principali componenti del sistema e a stabilire un livello iniziale di copertura del codice.

La baseline ha raggiunto un livello di copertura white-box pari a circa **62%**. Questo valore rappresenta il punto di partenza per le successive attività di estensione della test suite.

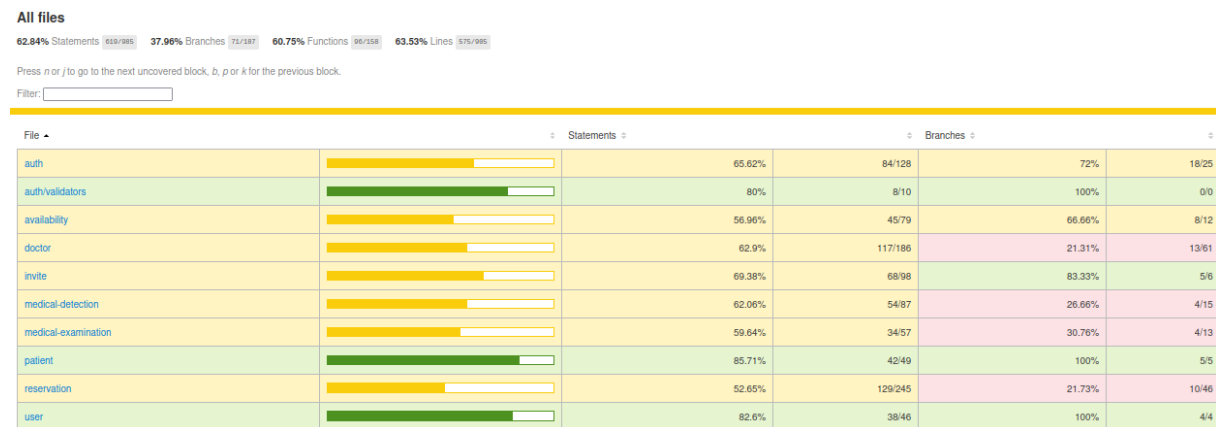


Figure 1.2: Baseline iniziale della copertura dei test

1.2.3 Crescita incrementale guidata dalle Change Request

Successivamente all'introduzione della baseline, la test suite è stata estesa progressivamente durante l'implementazione delle Change Request.

Per ogni modifica significativa introdotta nel sistema sono stati aggiunti nuovi test relativi ai moduli coinvolti, verificando la copertura del codice introdotto e assicurando la corretta integrazione con le funzionalità esistenti.

Per i moduli interessati dalle modifiche è stato perseguito l'obiettivo di raggiungere una copertura white-box prossima al **100%**, al fine di garantire un adeguato livello di verifica del comportamento del codice.

Questo approccio incrementale ha consentito di migliorare progressivamente la qualità del sistema e la robustezza della suite di test.

1.2.4 Integrazione con CI/CD e Sonar

Le attività di testing sono integrate nella pipeline di **Continuous Integration / Continuous Deployment (CI/CD)** del progetto.

Ad ogni aggiornamento del codice la pipeline esegue automaticamente:

- l'esecuzione completa della suite di test
- il calcolo delle metriche di code coverage
- l'analisi statica del codice tramite **Sonar**

Questo processo consente di monitorare in modo continuo la qualità del codice e di verificare automaticamente l'impatto delle modifiche introdotte nel sistema.

2 Architettura e Stato della Test Suite

Questa sezione descrive l'architettura della test suite del backend e lo stato della sua infrastruttura di testing. L'obiettivo è fornire una panoramica delle tecnologie utilizzate, della struttura dei test e delle metriche adottate per monitorare la qualità del codice durante l'evoluzione del sistema.

2.1 Framework e Strumenti di Testing

Il backend del progetto è sviluppato utilizzando il framework **NestJS** e adotta **Jest** come test runner principale per l'esecuzione dei test automatici. Jest viene utilizzato sia per i test unitari sia per i test di integrazione.

Per la simulazione delle richieste HTTP agli endpoint REST viene utilizzata la libreria **Supertest**. Questa libreria consente di inviare richieste HTTP direttamente all'applicazione NestJS inizializzata in ambiente di test, permettendo di verificare il comportamento del sistema dal livello di controller fino alla logica applicativa.

L'infrastruttura di testing utilizza inoltre il sistema di coverage integrato in Jest, basato su **Istanbul**, che consente di raccogliere metriche di copertura del codice durante l'esecuzione dei test.

2.2 Struttura della Test Suite

La test suite segue le convenzioni tipiche dei progetti NestJS e distingue due principali tipologie di test.

I **test unitari** sono collocati direttamente accanto ai file sorgente all'interno della directory **src**. Essi sono identificati dalla convenzione di naming ***.spec.ts**

Questo approccio consente di mantenere una stretta vicinanza tra il codice sorgente e i test che ne verificano il comportamento, facilitando la manutenzione e l'evoluzione della suite di test. I **test di integrazione** sono invece collocati nella directory dedicata **backend/test/**

Questi test seguono la convenzione ***.e2e-spec.ts**

Nonostante la nomenclatura utilizzata, tali test non costituiscono veri test end-to-end completi. Essi verificano il comportamento del sistema attraverso gli endpoint HTTP ma utilizzano repository mockati al posto di un database reale. Per questo motivo possono

essere più correttamente classificati come **integration tests con caratteristiche gray-box**, in quanto testano l'interazione tra più componenti applicativi mantenendo comunque un certo livello di isolamento dall'infrastruttura esterna.

2.3 Uso del Mocking

La strategia di testing adottata fa largo uso di **mock** per isolare la logica applicativa dalle dipendenze esterne. In particolare, nei test unitari e nei test di integrazione i repository di TypeORM vengono sostituiti con oggetti mock tramite le funzionalità di mocking offerte da Jest.

Questo approccio consente di:

- ridurre significativamente il tempo di esecuzione dei test;
- isolare la logica di business dalle componenti infrastrutturali;
- controllare in modo deterministico il comportamento delle dipendenze.

Grazie a questa strategia, i test non richiedono la presenza di un database reale e possono essere eseguiti rapidamente sia in ambiente locale sia nelle pipeline di integrazione continua.

2.4 Metriche di Coverage

La copertura del codice viene monitorata tramite le metriche standard fornite da Istanbul:

- Statement Coverage
- Branch Coverage
- Function Coverage
- Line Coverage

Come già citato, durante l'introduzione iniziale della test suite è stata definita una **baseline di coverage pari a circa il 60%**. Questa baseline rappresenta il punto di partenza della strategia di testing adottata nel progetto.

L'obiettivo del progetto è incrementare progressivamente la copertura del codice attraverso l'introduzione di nuovi test in occasione delle diverse Change Request, con particolare attenzione alla copertura delle logiche applicative più critiche.

Alcune categorie di file vengono deliberatamente escluse dal calcolo della coverage in quanto contengono principalmente configurazioni o strutture dati prive di logica applicativa significativa. Tra questi rientrano tipicamente:

- moduli di NestJS (*.module.ts);
- Data Transfer Object (*.dto.ts);
- entità di persistenza (*.entity.ts);
- file di configurazione dell'applicazione.

2.5 Integrazione con CI/CD e Analisi Statica

L'esecuzione dei test è integrata all'interno della pipeline di integrazione continua del progetto, implementata tramite **GitHub Actions**. Ad ogni apertura o aggiornamento di una Pull Request viene eseguita automaticamente la suite di test per verificare la correttezza delle modifiche introdotte.

Parallelamente, il progetto utilizza **SonarCloud** per l'analisi statica del codice. Questo strumento consente di monitorare diversi indicatori di qualità del software, tra cui:

- copertura del codice;
- code smells;
- vulnerabilità di sicurezza;
- duplicazione del codice.

Riportiamo qui il report riepilogativo al termine del progetto

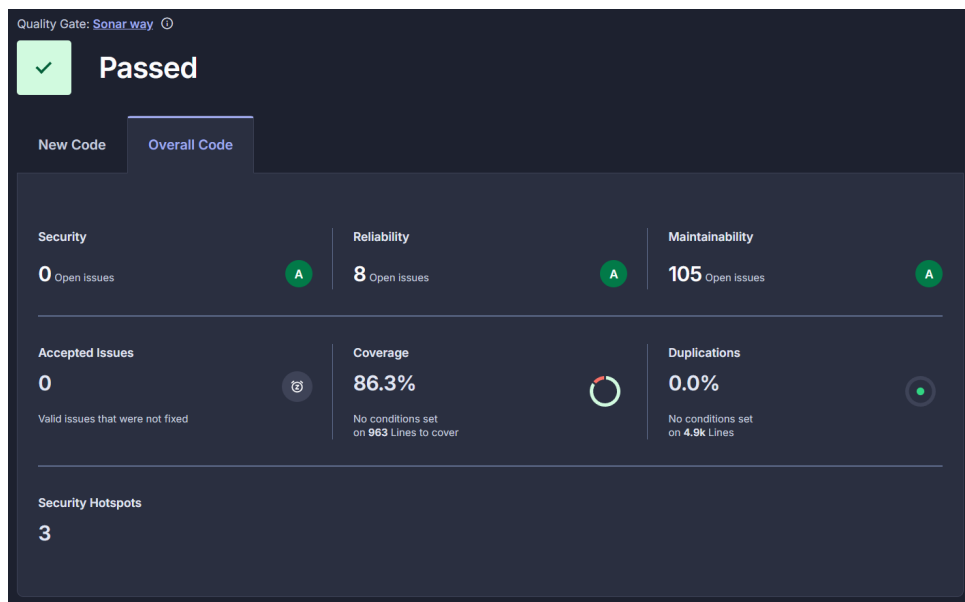


Figure 2.1: Report SonarQube

3 Strategia di Testing e Regression Testing

Questa sezione illustra le strategie di testing adottate in concomitanza con le modifiche evolutive apportate al software (Change Request) e le relative pratiche di regression testing per assicurare che il codice preesistente non subisca regressioni.

3.1 Processo di testing dopo una Change Request

3.1.1 Identificazione delle aree impattate

A seguito di ogni Change Request (CR), il primo passo consiste nell'individuazione delle **aree impattate** dal cambiamento. Questo avviene analizzando le dipendenze nei moduli, sia lato **backend** (es. dipendenze tra i vari `*.module.ts` di NestJS) che lato **frontend** (es. dipendenze nei componenti React all'interno della cartella `frontend/src`).

3.1.2 Moduli e layer coinvolti

Si valutano i layer architetturali coinvolti dalla modifica:

- **Controller, Service, Entity:** Tipicamente coinvolti per le modifiche all'interno della cartella `backend/src/`.
- **Componenti UI e State Management:** Coinvolti quando le CR interessano la User Experience e le viste React, le cui modifiche risiedono nella directory `frontend/`.

L'analisi dei moduli assicura che vengano aggiornati o creati i relativi file di test per coprire i nuovi scenari introdotti, focalizzando lo sforzo direttamente sulle porzioni di codice interessate.

3.2 Strategia di regression testing

Nel progetto le attività di regression testing sono state progettate con l'obiettivo di garantire che le modifiche introdotte tramite Change Request non compromettano il corretto funzionamento delle funzionalità già presenti nel sistema.

Considerata la dimensione contenuta del progetto e il tempo di esecuzione ridotto della test suite, è stata adottata una strategia di **esecuzione completa dei test ad ogni modifica**. In particolare, ad ogni aggiornamento del codice vengono eseguiti tutti i test disponibili, evitando quindi meccanismi di selezione o prioritizzazione dei test.

Questa scelta consente di semplificare il processo di verifica e di massimizzare l'affidabilità dei controlli, riducendo il rischio che eventuali regressioni non vengano rilevate.

Nel dettaglio, la strategia di testing prevede:

- **Esecuzione completa della test suite:** ad ogni modifica del codice vengono eseguiti tutti i test unitari e di integrazione (ad esempio tramite i comandi `npm run test` e `npm run test:e2e` per le suite presenti nella directory `backend/`).
- **Test end-to-end sui flussi principali:** i test end-to-end, situati ad esempio in `backend/test/`, coprono i flussi operativi più critici del sistema e vengono mantenuti aggiornati durante l'evoluzione del progetto.
- **Monitoraggio della copertura dei test:** i report di code coverage (generati nella directory `backend/coverage/`) vengono analizzati periodicamente per individuare componenti con copertura insufficiente e guidare l'introduzione di nuovi test.
- **Integrazione con CI/CD:** la pipeline di integrazione continua esegue automaticamente la suite di test e l'analisi statica tramite **SonarCloud**. Nei branch protetti (ad esempio `develop` e `main`) il superamento del Quality Gate rappresenta una condizione necessaria per l'integrazione delle modifiche.

Questo approccio garantisce un controllo continuo della qualità del codice e consente di individuare rapidamente eventuali regressioni introdotte durante le attività di manutenzione evolutiva.

4 Criteri di Adeguatezza del Testing

Per valutare la qualità e l'efficacia della test suite sviluppata per il sistema, sono stati definiti specifici **criteri di adeguatezza del testing**. Tali criteri stabiliscono quando la copertura dei test può essere considerata sufficiente per garantire un adeguato livello di affidabilità del software.

Sono stati adottati due approcci complementari:

- **White-Box Testing**, basato sulla struttura interna del codice
- **Black-Box Testing**, basato sul comportamento osservabile del sistema

L'uso combinato di queste due strategie consente di ottenere una copertura più completa sia della logica implementata nel codice sia delle funzionalità offerte dal sistema.

4.1 Criteri White-Box (Testing White Box)

Il **White-Box Testing** consiste nella progettazione dei casi di test analizzando la struttura interna del codice sorgente. L'obiettivo principale è verificare che le diverse parti del programma siano effettivamente eseguite durante l'esecuzione dei test.

Per valutare l'adeguatezza della test suite sono state utilizzate metriche di **code coverage**, che misurano la percentuale di codice eseguita durante i test.

4.1.1 Coverage minimo dichiarato

Per il progetto è stato definito un **livello minimo di copertura del codice**, considerato sufficiente per garantire un buon livello di verifica della logica applicativa.

Statement Coverage Target

La **statement coverage** misura la percentuale di istruzioni del programma eseguite almeno una volta durante l'esecuzione dei test.

Per il progetto è stato definito il seguente obiettivo:

- Statement Coverage $\geq 80\%$

Questo criterio garantisce che la maggior parte delle istruzioni del codice venga eseguita almeno una volta durante la test execution.

Branch Coverage Target

La **branch coverage** misura la percentuale di rami decisionali del codice (ad esempio condizioni **if**, **switch**, **while**) che vengono effettivamente percorsi dai test.

Per il progetto è stato stabilito il seguente target:

- Branch Coverage $\geq 70\%$

Questo tipo di copertura è particolarmente importante perché verifica che vengano testati sia i percorsi **veri** sia quelli **falsi** delle condizioni logiche.

Focus sui servizi core

L'attenzione principale della copertura white-box è stata posta sui **servizi core del sistema**, ovvero:

- servizi di autenticazione
- servizi di gestione utenti
- servizi di accesso ai dati
- servizi contenenti logica applicativa critica

Queste componenti rappresentano le parti più sensibili del sistema e richiedono quindi un livello di copertura più elevato rispetto ai componenti puramente infrastrutturali.

4.1.2 Dove è più rilevante la Branch Coverage

La **branch coverage** è particolarmente rilevante nei componenti del sistema che contengono logica decisionale complessa.

In particolare, essa è stata applicata con maggiore attenzione nei seguenti casi.

Servizi con logica decisionale complessa

I servizi che implementano **logiche di business articolate** possono presentare numerosi percorsi di esecuzione alternativi.

In questi casi, la branch coverage permette di verificare che i test coprano tutte le possibili decisioni logiche presenti nel codice.

Autenticazione

I moduli di autenticazione contengono spesso diversi rami decisionali, ad esempio:

- credenziali valide / non valide
- token valido / scaduto
- utente autorizzato / non autorizzato

Una copertura adeguata di questi rami è fondamentale per garantire la sicurezza del sistema.

4.2 Criteri Black-Box e Integration Testing

Oltre ai test white-box a livello di unità, il progetto include una serie di **test di integrazione** che verificano il comportamento del sistema attraverso l'interfaccia HTTP esposta dal backend.

Questi test sono implementati come **End-to-End (E2E) test** e simulano richieste HTTP reali verso gli endpoint dell'applicazione. L'obiettivo è verificare il corretto funzionamento dell'interazione tra i principali componenti del sistema, tra cui:

- Controller
- Service
- Guard
- Interceptor

I test sono implementati utilizzando la libreria **supertest**, che permette di simulare chiamate HTTP verso l'applicazione NestJS durante l'esecuzione della suite di test.

Dal punto di vista metodologico, questi test possono essere considerati una forma di **Gray-Box Testing**.

Infatti:

- il sistema viene testato attraverso l'interfaccia HTTP pubblica (comportamento osservabile, tipico del black-box testing)
- ma la configurazione dell'ambiente di test utilizza **mock dei repository** per isolare la logica applicativa

Questo approccio consente di ottenere un buon compromesso tra:

- realismo dei test
- velocità di esecuzione
- isolamento dei componenti

4.2.1 Copertura funzionale tramite test di integrazione

I test di integrazione sono progettati per coprire i principali flussi funzionali del sistema. In particolare, ogni suite di test verifica uno o più **casi d'uso principali** dell'applicazione. Per ciascun flusso vengono generalmente considerati:

- uno **scenario positivo**, in cui il sistema si comporta correttamente
- uno o più **scenari negativi**, che verificano la gestione degli errori

Questo consente di validare il comportamento complessivo dell'applicazione dal punto di vista dell'API.

4.2.2 Strategia di mocking

Nonostante siano test di integrazione, il backend utilizza dei **mock dei repository** al posto dell'accesso reale al database.

I repository TypeORM vengono sostituiti da oggetti mockati, ad esempio:

```
const mockUserRepository = {  
  findOne: jest.fn(),  
  save: jest.fn()  
};
```

Questo approccio offre diversi vantaggi:

- **Velocità**: i test non richiedono l'avvio di un database reale
- **Isolamento**: è possibile controllare il comportamento dei repository
- **Riproducibilità**: i test non dipendono dallo stato del database

4.2.3 Esempio di flusso testato

Un esempio significativo è rappresentato dalla suite di test dedicata al **flusso di autenticazione con autenticazione a due fattori (2FA)**.

Questo test verifica l'intero processo di autenticazione, includendo:

- autenticazione con credenziali valide
- verifica della presenza del requisito di autenticazione a due fattori
- invio e validazione del codice OTP
- gestione degli errori in caso di codice non valido

Attraverso questi scenari viene verificata l'integrazione tra:

- `AuthController`
- `AuthService`
- `TwoFactorService`
- meccanismi di autenticazione e gestione delle sessioni

4.2.4 Esecuzione dei test

I test di integrazione sono collocati nella cartella:

`backend/test/`

e possono essere eseguiti tramite il comando:

```
npm run test:e2e
```

Durante l'esecuzione, il framework di testing **Jest** avvia l'applicazione in modalità di test e utilizza **Supertest** per inviare richieste HTTP simulate agli endpoint dell'API.

5 Infrastruttura di Test e Qualità del Progetto

Il presente capitolo descrive e documenta l'infrastruttura di test, gli strumenti di analisi statica e il processo di integrazione continua adottati nel progetto. Sono analizzati gli strumenti utilizzati, la configurazione degli ambienti di testing, nonché le metodologie implementate per garantire la qualità del codice nel backend (NestJS) e nel frontend (React).

5.1 Test Runner e TypeScript

5.1.1 Test Runner

Per l'esecuzione dei test automatizzati, il progetto utilizza **Jest** (versione 29.7.0) come framework di test principale. Jest è stato scelto per la sua integrazione nativa con TypeScript e la facilità di configurazione per ambienti Node.js come il backend del progetto.

5.1.2 Configurazione TypeScript

La configurazione di TypeScript è gestita tramite il pacchetto **ts-jest** (versione 29.4.1), che permette di utilizzare Jest per testare codice TypeScript senza necessità di compilazioni intermedie. Un file di configurazione dedicato, `backend/src/jest.config.ts`, definisce i percorsi di origine (`<rootDir>/src`) e la risoluzione degli alias tramite `moduleNameMapper`, che consente di utilizzare gli alias come `src/` per una gestione più chiara dei percorsi.

5.1.3 Esecuzione dei Test

I test vengono eseguiti tramite i seguenti comandi di Jest:

- `npm test`: Esegue tutti i test unitari definiti nel progetto.
- `npm run test:watch`: Modalità interattiva per lo sviluppo dei test.
- `npm run test:e2e`: Esegue i test di integrazione e end-to-end situati nella cartella `test/`.

5.2 Coverage Tool (Istanbul/Jest)

5.2.1 Motore di Coverage

Il motore di coverage utilizzato è quello nativo di **Jest**, che si basa su **Istanbul** per la generazione dei report di copertura del codice. La generazione dei report di coverage permette di monitorare in modo dettagliato quali parti del codice sono coperte dai test.

5.2.2 Specifiche di Coverage

Le seguenti metriche di coverage sono monitorate durante l'esecuzione dei test:

- **Line coverage**: Percentuale di linee di codice coperte dai test.
- **Statement coverage**: Percentuale di istruzioni eseguite nei test.
- **Branch coverage**: Percentuale di rami condizionali coperti dai test.
- **Function coverage**: Percentuale di funzioni eseguite nei test.

La configurazione di Jest nel file `package.json` esclude correttamente i file di configurazione, i DTO, le entità e altri file boilerplate (`.module.ts`, `.dto.ts`, `.entity.ts`, ecc.), in modo da fornire una metrica di coverage che si concentri sulla logica di business del progetto.

5.2.3 Generazione del Report

Il report di coverage viene generato tramite il comando:

```
npm run test:cov
```

Il report completo viene salvato nella cartella `backend/coverage/`, permettendo agli sviluppatori di visualizzare il livello di copertura in modo semplice e immediato.

5.3 Continuous Integration (CI)

5.3.1 Piattaforma di Integrazione Continua

L'automazione del processo di testing è gestita tramite **GitHub Actions**, che permette di eseguire i test automaticamente in seguito a ogni push o pull request. L'integrazione continua è configurata attraverso due workflow principali:

- **CI - Develop Integration** (`ci-develop.yml`): Questo workflow è attivato ogni volta che viene effettuato un push sul branch `develop`.
- **CI - Pull Request** (`ci-pr.yml`): Questo workflow viene attivato all'apertura di una pull request verso il branch `develop`.

5.3.2 Esecuzione dei Test in CI

Nel processo di CI, vengono eseguiti i seguenti passaggi:

- Installazione delle dipendenze tramite `npm ci`.
- Esecuzione dei test tramite `npm run test`.
- Esecuzione del build del progetto tramite `npm run build`.

Tutti i risultati dei test e i report di coverage vengono inviati automaticamente a **SonarCloud** per la visualizzazione centralizzata, permettendo un monitoraggio continuo della qualità del codice.

5.4 Analisi Statica (SonarCloud/SonarQube)

5.4.1 Piattaforma di Analisi Statica

Il progetto è integrato con **SonarCloud** tramite l'azione ufficiale **SonarSource/sonarcloud-github-action**. SonarCloud fornisce una panoramica della qualità del codice, monitorando metriche critiche e identificando eventuali problemi nel codice.

5.4.2 Configurazione di SonarCloud

La configurazione di SonarCloud è gestita tramite il file `backend/sonar-project.properties`, dove vengono definiti la `projectKey` e l'`organization`. In questo file sono anche configurati i parametri per l'invio dei dati di qualità.

5.4.3 Metriche di Qualità Monitorate

SonarCloud monitora continuamente diverse metriche di qualità del codice:

- **Bugs e Vulnerabilities** (Sicurezza)
- **Code Smells** (Manutenibilità)
- **Security Hotspots**

- **Code Coverage** (proveniente dai report di Jest)
- **Duplicazione del codice**

Il feedback su queste metriche viene fornito direttamente su GitHub, dove viene bloccato il merge di una pull request se il Quality Gate di SonarCloud non è soddisfatto.

5.5 Gestione Dati di Test

5.5.1 Mock e Fixtures

Per garantire la velocità e il determinismo nei test, i test di integrazione (`.e2e-spec.ts`) e unitari utilizzano mock per i repository del database e i servizi esterni. Ad esempio, in `auth-mfa.e2e-spec.ts` viene creato un `mockUserRepository` per simulare l'accesso ai dati senza la necessità di un database reale.

5.5.2 Database

L'applicazione utilizza **PostgreSQL** tramite **Supabase** negli ambienti di sviluppo e produzione. Per i test, vengono utilizzati **database mockati** o **in-memory DB** per consentire una riproducibilità più rapida dei test e per isolare i test dal database di produzione.

5.5.3 In-memory DB

I test sono configurati per operare in isolamento, iniettando versioni mockate delle entità TypeORM tramite il pacchetto `@nestjs/testing`. Questo approccio consente di eseguire i test in modo rapido e indipendente dal database effettivo, migliorando la velocità e la determinazione dei test.

5.6 Riproducibilità dei Test

5.6.1 Locale vs CI

I test sono 100% riproducibili sia in ambiente locale che in CI. Il workflow in CI utilizza gli stessi script e configurazioni di `package.json` utilizzati localmente, garantendo una completa coerenza tra i due ambienti.

5.6.2 Variabili d'Ambiente

Le variabili di ambiente vengono gestite in modo coerente sia in ambiente locale che in CI. In particolare:

- In locale, viene utilizzato il file `.env`, gestito tramite il pacchetto `@nestjs/config`.
- In CI, le variabili sensibili, come `SONAR_TOKEN` e `GITHUB_TOKEN`, sono iniettate tramite **GitHub Secrets**.

Il pacchetto **cross-env** viene utilizzato per garantire che le variabili d'ambiente siano gestite correttamente su tutti i sistemi operativi (Windows, macOS, Linux), eseguendo comandi come `cross-env STAGE=dev jest`.

6 Assessment e Miglioramenti Futuri

In quest'ultimo capitolo viene effettuato un bilancio delle attività di testing fino ad ora pianificate ed eseguite, confrontando i traguardi raggiunti con le aree del sistema, afferenti sia all'ambiente **backend** che **frontend**, ritenute ancora perfettibili. L'obiettivo è tracciare un percorso di consolidamento progressivo dell'infrastruttura di Quality Assurance del progetto ISTA2026.

6.1 Risultati ottenuti

Le fasi operative, la metodologia impiegata e le pratiche adottate hanno generato molteplici risultati positivi che consolidano l'affidabilità del sistema:

- **Introduzione baseline:** È stata creata con successo una base di testing solida per il progetto (raggiungendo inizialmente circa il 60% di copertura) concentrata principalmente nella root **backend**/. Questo ha sancito il passaggio da un progetto privo di automazioni di garanzia a un ecosistema validato.
- **Incremento progressivo coverage:** Guidati dall'approccio guidato dalle Change Request, si è verificato un aumento continuo della test coverage per statement e branch all'evolvere del codice, ponendo enfasi rigorosa sui moduli core dell'applicazione (es. Autenticazione e Prenotazioni).
- **Integrazione CI/CD:** Automatizzando l'esecuzione dei test tramite GitHub Actions (innescate su branch come **develop** o sulle Pull Request), è stato creato un efficace e tempestivo perimetro di rilevazione anomalie che ostacola il merge di codice non adeguato o regredito.

6.2 Limiti attuali e possibili miglioramenti

Nonostante il miglioramento significativo della qualità del codice e l'introduzione di una infrastruttura di testing automatizzata, la test suite presenta ancora alcuni limiti, tipici di progetti sviluppati inizialmente senza una strategia di testing strutturata.

In primo luogo, alcune componenti del sistema risultano meno coperte rispetto ai servizi principali del backend. In particolare, alcune parti secondarie dell'applicazione e diversi

componenti del **frontend** non dispongono ancora di una suite di test dedicata. L'attuale infrastruttura di testing è infatti concentrata principalmente sul backend e sui servizi che implementano la logica applicativa principale.

Un secondo limite riguarda la **branch coverage**. Sebbene la copertura delle istruzioni sia complessivamente soddisfacente, alcuni rami decisionali più complessi — ad esempio quelli legati alla gestione delle eccezioni o alla validazione di input non validi — non risultano ancora completamente coperti dai test unitari.

Infine, l'attuale suite di test di integrazione, situata nella directory **backend/test/**, verifica principalmente il comportamento delle API del backend. Sebbene questi test consentano di controllare correttamente l'interazione tra controller, servizi e altri componenti applicativi, non sono presenti test che simulino l'interazione completa tra frontend e backend tramite l'interfaccia utente reale.

Nel complesso, possibili miglioramenti futuri includono l'estensione della copertura dei test unitari ai rami decisionali meno esplorati, l'introduzione di test aggiuntivi per alcuni servizi secondari e, ove necessario, la definizione di ulteriori scenari di test per i flussi applicativi principali.

7 Change Request #1

7.1 Stato del sistema di testing prima della Change Request

Prima dell'introduzione della presente Change Request, il progetto disponeva già di una suite di **unit test** utilizzata per verificare la logica dei principali servizi applicativi.

Gli unit test erano collocati all'interno delle directory dei rispettivi moduli applicativi, seguendo il pattern `*.spec.ts`. Il framework utilizzato per l'esecuzione dei test è **Jest**, configurato per l'ambiente **TypeScript**.

Per isolare la logica applicativa dalle dipendenze esterne venivano utilizzati **mock**, in particolare per i repository gestiti tramite TypeORM.

Prima dell'introduzione della Change Request la copertura complessiva del codice risultava circa pari al 60%

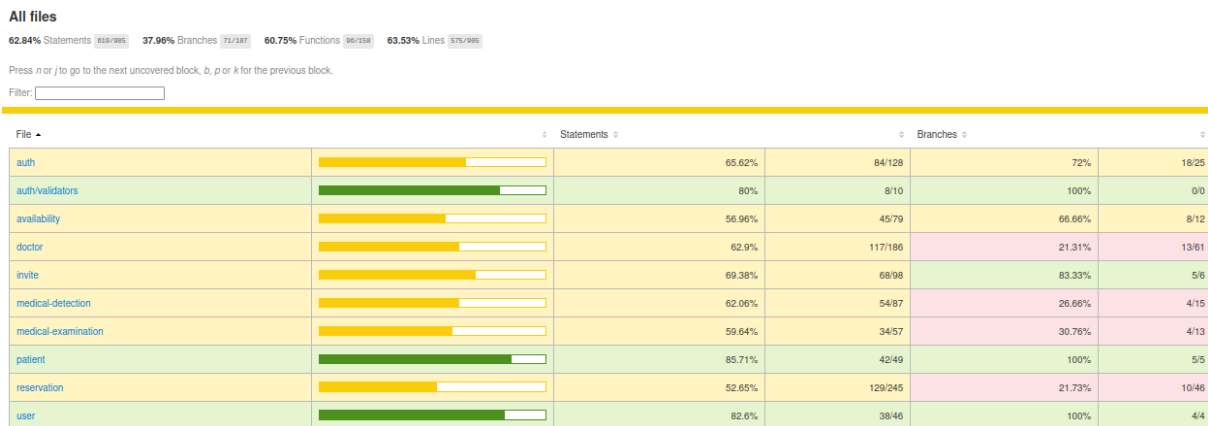


Figure 7.1: Copertura di test iniziale

In questa fase non erano presenti **test di integrazione o end-to-end**; la verifica del sistema avveniva esclusivamente tramite unit test.

7.2 Test introdotti per la Change Request

L'introduzione dell'autenticazione a due fattori ha richiesto l'estensione della suite di test per verificare il corretto funzionamento del nuovo flusso di autenticazione.

Sono stati introdotti test di integrazione/end-to-end dedicati alla verifica del flusso completo di autenticazione multi-factor. Il principale file di test introdotto è:

`backend/test/auth-mfa.e2e-spec.ts`

Questo test inizializza un'istanza dell'applicazione NestJS includendo i componenti necessari alla gestione dell'autenticazione, tra cui:

- `AuthController`
- `AuthService`
- `TwoFactorService`

Per evitare l'utilizzo di un database reale durante l'esecuzione dei test, i repository di TypeORM vengono sostituiti tramite **mock**.

Il file di test verifica i principali scenari del nuovo flusso di autenticazione:

- **Sign-in standard (2FA disabilitato)** Verifica che, in assenza di 2FA attiva, l'inserimento di credenziali valide produca direttamente l'emissione dei token di sessione tramite cookie `HttpOnly`.
- **Sign-in con 2FA abilitato** Verifica che, quando l'utente ha il 2FA attivo, il sistema non restituisca immediatamente i token ma una risposta contenente il flag `requires2fa: true` e un identificatore di challenge.
- **Verifica OTP con codice valido** Simula l'invio di un codice OTP corretto insieme al `challengeId`. Il sistema valida il codice e restituisce i token di accesso.
- **Verifica OTP con codice non valido** Verifica che l'invio di un codice OTP errato produca una risposta `401 Unauthorized` e che il contatore dei tentativi venga incrementato.

Parallelamente sono stati estesi gli **unit test** per coprire i nuovi componenti introdotti dalla Change Request, in particolare la logica di generazione e verifica dei codici OTP.

Grazie all'introduzione di questi test, la copertura degli unit test è stata portata a circa 100% per i moduli interessati dalla modifica.

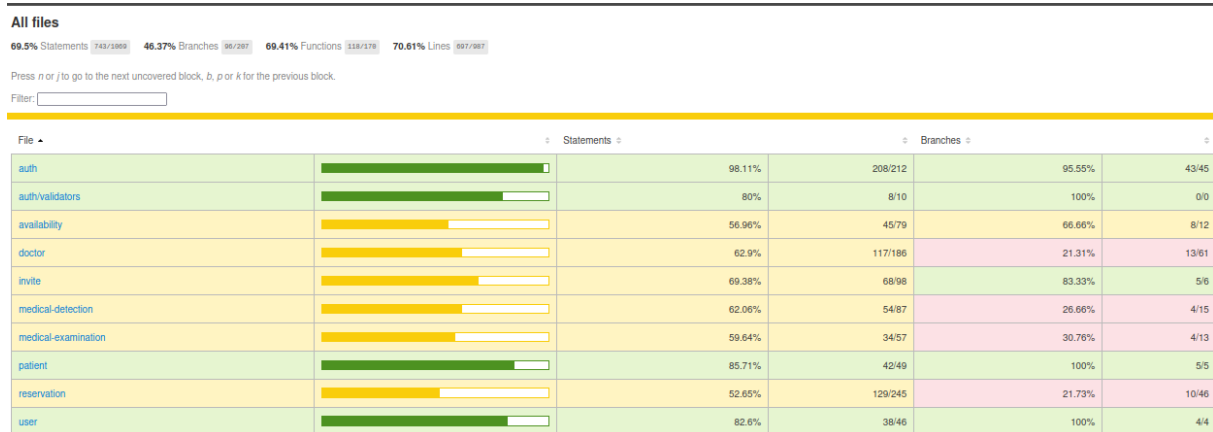


Figure 7.2: Copertura di test dopo la Change Request

7.3 Strategia di regression testing

Dopo l'implementazione della Change Request è stata rieseguita l'intera suite di test del progetto.

L'esecuzione dei test non ha evidenziato anomalie o regressioni nel comportamento del sistema. Tutti i test esistenti e quelli introdotti per la gestione del 2FA sono stati eseguiti correttamente e hanno prodotto esito positivo.

Questo ha confermato che l'introduzione dell'autenticazione a due fattori non ha compromesso il funzionamento delle funzionalità già presenti nel sistema.

7.4 Risultati dell'esecuzione dei test

Al termine dell'implementazione della Change Request, l'intera suite di test è stata eseguita sia in ambiente locale sia attraverso la pipeline di integrazione continua del progetto.

Il progetto utilizza una pipeline di **Continuous Integration** che esegue automaticamente i test ad ogni aggiornamento del repository.

L'esecuzione della suite ha prodotto i seguenti risultati:

- tutti i test sono stati eseguiti con successo;
- i nuovi scenari di autenticazione con 2FA sono stati verificati correttamente;
- non sono state rilevate regressioni nelle funzionalità di autenticazione preesistenti.

7.5 Coverage e miglioramento della qualità

L'introduzione dei nuovi test ha migliorato la copertura del codice, in particolare per quanto riguarda il modulo di autenticazione e i componenti associati alla gestione del 2FA.

La tabella seguente riporta il confronto tra la copertura prima e dopo l'introduzione della Change Request per il modulo impattato.

Metrica	Prima della CR	Dopo la CR
Statement Coverage	65%	98%
Branch Coverage	72%	100%
Line Coverage	65%	99%

L'incremento della copertura è principalmente dovuto all'estensione degli unit test e all'introduzione di test di integrazione dedicati al flusso di autenticazione multi-factor.

7.6 Valutazione finale del testing della CR

L'introduzione dei test associati alla Change Request ha consentito di verificare il corretto funzionamento del nuovo sistema di autenticazione a due fattori.

Gli unit test garantiscono una copertura completa della logica introdotta, mentre il test di integrazione verifica il comportamento del flusso di autenticazione nel suo complesso.

L'esecuzione dell'intera suite di test ha inoltre confermato l'assenza di regressioni nelle funzionalità esistenti, garantendo una corretta integrazione della nuova funzionalità con il resto del sistema.

8 Change Request #2

8.1 Stato del sistema di testing prima della Change Request

Prima dell'introduzione della Change Request #2, la suite di test del progetto era composta principalmente da unit test che coprivano la logica dei servizi applicativi così come prima della Change Request 1.

In particolare, il modulo di gestione delle prenotazioni presentava una copertura parziale, con alcune porzioni di logica non ancora verificate dai test. Le lacune riguardavano principalmente:

- casi limite nella creazione delle prenotazioni;
- gestione degli errori in alcuni metodi del service;
- validazioni relative al tipo di visita e alla disponibilità degli slot;
- controlli sui ruoli nei metodi del controller.

Queste lacune sono state identificate come parte dell'attività di estensione della suite di test durante l'implementazione della Change Request.

8.2 Test introdotti per la Change Request

Per verificare il comportamento complessivo del sistema durante il nuovo processo di onboarding sicuro dei pazienti, è stato introdotto un test di integrazione che copre il flusso completo dell'endpoint `POST /doctor/invite`.

Il test ha verificato i seguenti scenari:

- **Creazione del paziente:** Quando un medico inserisce un nuovo paziente, il sistema crea correttamente l'account utente associato.
- **Invio della password temporanea:** La password temporanea viene generata e inviata correttamente al paziente via email.

- **Impostazione del flag `mustChangePassword`:** Il flag `mustChangePassword` viene impostato a `true` per il primo accesso del paziente.
- **Verifica duplicati:** Il sistema verifica la presenza di duplicati in base ai dati del paziente (email, telefono, codice fiscale).
- **Gestione dei dati non validi:** Il sistema rifiuta correttamente richieste malformate e restituisce il messaggio di errore appropriato.

Tutti gli scenari sono stati testati con esito positivo, confermando che l'endpoint si comporta come previsto.

È stato inoltre estesa la suite di test unitari in modo significativo. In particolare sono stati aggiornati ed ampliati i file di test: `reservation.service.spec.ts` e `reservation.controller.spec.ts`

L'obiettivo principale di questa attività è stato raggiungere una copertura completa della logica applicativa introdotta o modificata dalla Change Request.

Sono stati aggiunti test per verificare diversi scenari del modulo di prenotazione, tra cui:

- gestione dei casi in cui non sono presenti prenotazioni future;
- verifica dei filtri applicati al recupero delle prenotazioni passate;
- logica di validazione per le prenotazioni di tipo `FIRST_VISIT`;
- gestione degli errori quando una prenotazione non è presente nel sistema;
- verifica dei parametri di paginazione di default;
- validazione dei controlli sui ruoli negli endpoint del controller.

Grazie all'introduzione di questi test, è stato possibile migliorare significativamente la copertura del codice del modulo prenotazioni.

I risultati finali di copertura sono:

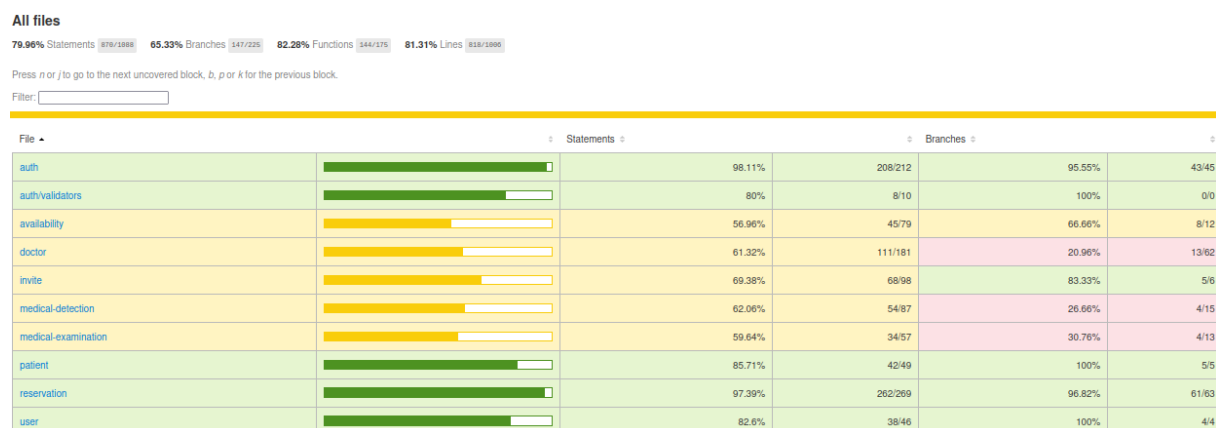


Figure 8.1: Copertura di test dopo la Change Request 2

Questo risultato garantisce che tutta la logica implementata nel modulo di prenotazione sia coperta da test automatici.

8.3 Strategia di regression testing

Dopo l'implementazione della Change Request è stata rieseguita l'intera suite di test del progetto al fine di verificare che le modifiche introdotte nel modulo di prenotazioni non avessero impattato il comportamento delle funzionalità preesistenti.

L'esecuzione dei test ha permesso di identificare eventuali incompatibilità tra la nuova implementazione e i test legacy, in particolare nelle componenti interessate dal refactoring del `ReservationService` e del `ReservationController`.

8.4 Risultati dell'esecuzione dei test

Durante la prima esecuzione della suite di test sono emersi alcuni fallimenti nei test legacy relativi al modulo di prenotazioni.

Le cause principali erano riconducibili a modifiche introdotte dal refactoring necessario per supportare il nuovo flusso di creazione delle prenotazioni da parte del medico, in particolare:

- utilizzo di un nome metodo non più presente nel service;
- disallineamento dei mock utilizzati nei test del controller;
- nuove dipendenze introdotte nel service non dichiarate nel modulo di test.

Dopo l'analisi delle cause e l'aggiornamento dei test e dei relativi mock, l'intera suite è stata rieseguita con esito positivo.

Tutti i test risultano ora correttamente eseguiti e compatibili con la nuova logica introdotta dalla Change Request.

8.5 Coverage e miglioramento della qualità

L'estensione della suite di test ha portato ad un miglioramento significativo della copertura del modulo di prenotazioni.

In particolare è stato possibile raggiungere:

- copertura completa del service di gestione delle prenotazioni;

- copertura completa delle principali logiche del controller;
- verifica di numerosi casi limite precedentemente non testati.

La tabella seguente riporta il confronto tra la copertura prima e dopo l'introduzione della Change Request per il modulo impattato.

Metrica	Prima della CR	Dopo la CR
Statement Coverage	52%	97%
Branch Coverage	21%	97%
Line Coverage	53%	98%

Questo miglioramento contribuisce a ridurre il rischio di regressioni future e garantisce una maggiore affidabilità del sistema.

8.6 Valutazione finale del testing della CR

L'attività di testing associata alla Change Request ha permesso di verificare il corretto funzionamento della nuova funzionalità di creazione delle prenotazioni da parte del medico.

L'estensione degli unit test ha consentito di coprire completamente la logica del modulo di prenotazioni, mentre l'attività di regression testing ha garantito la compatibilità della modifica con il comportamento preesistente del sistema.

Nel complesso, la suite di test risultante fornisce un elevato livello di confidenza sul corretto funzionamento della funzionalità introdotta.

9 Change Request #3

9.1 Stato del sistema di testing prima della Change Request

Prima dell'introduzione della Change Request #3, il progetto disponeva già di una suite di test automatizzati composta da unit test e da test di integrazione introdotti nelle Change Request precedenti.

In particolare, i moduli di autenticazione e prenotazione risultavano già coperti da test dedicati, mentre il nuovo flusso di onboarding sicuro dei pazienti non era ancora presente nella codebase e non disponeva quindi di una copertura specifica.

La modifica introdotta dalla CR #3 ha avuto un impatto rilevante sul modulo **Doctor**, in quanto ha comportato:

- l'introduzione di una nuova logica di onboarding lato backend;
- la rimozione del precedente flusso basato su **Invite**;
- la creazione immediata dell'account utente associato al paziente;
- la generazione e l'invio di credenziali temporanee;
- la gestione del flag `mustChangePassword`.

Di conseguenza, si è reso necessario estendere la suite di test per coprire in modo esplicito sia la nuova logica di business sia il nuovo endpoint introdotto per il medico.

9.2 Test introdotti per la Change Request

Per validare la nuova funzionalità sono stati introdotti sia **unit test** sia **test di integrazione/end-to-end** dedicati al modulo **Doctor**. In particolare, i file coinvolti sono:

```
doctor.service.spec.ts
```

```
doctor.controller.spec.ts
```

cr3-invite.e2e-spec.ts

Gli unit test sono stati progettati per coprire integralmente la logica introdotta o modificata nel `DoctorService` e nel `DoctorController`, raggiungendo una copertura completa delle righe di codice per entrambi i file.

Nel caso del `DoctorController`, i test verificano principalmente il corretto inoltro dei parametri verso il service sottostante, includendo anche il nuovo metodo `createInvite`, che rappresenta il punto di ingresso del nuovo flusso di onboarding. Sono inoltre verificati i casi relativi a paginazione, recupero pazienti, aggiornamento ed eliminazione, così da garantire che le modifiche introdotte non abbiano alterato i comportamenti preesistenti.

Per quanto riguarda il `DoctorService`, i test di unità coprono in modo esteso i principali rami decisionali introdotti dalla CR, in particolare nel metodo `createInvite`. Tra gli scenari verificati rientrano:

- rilevazione di duplicati su email, telefono o codice fiscale;
- gestione di conflitti emersi durante la creazione utente;
- errori durante l'hashing della password temporanea;
- fallimento dell'associazione tra paziente e utente;
- gestione di errori nell'invio email;
- esecuzione corretta del flusso completo di creazione del paziente e del relativo account utente.

Particolare attenzione è stata posta alla validazione dei casi di errore, così da coprire sia i percorsi di successo sia i rami eccezionali della nuova logica applicativa. Grazie all'introduzione di questi test, è stato possibile migliorare significativamente la copertura del codice del modulo prenotazioni. I risultati finali di copertura sono:

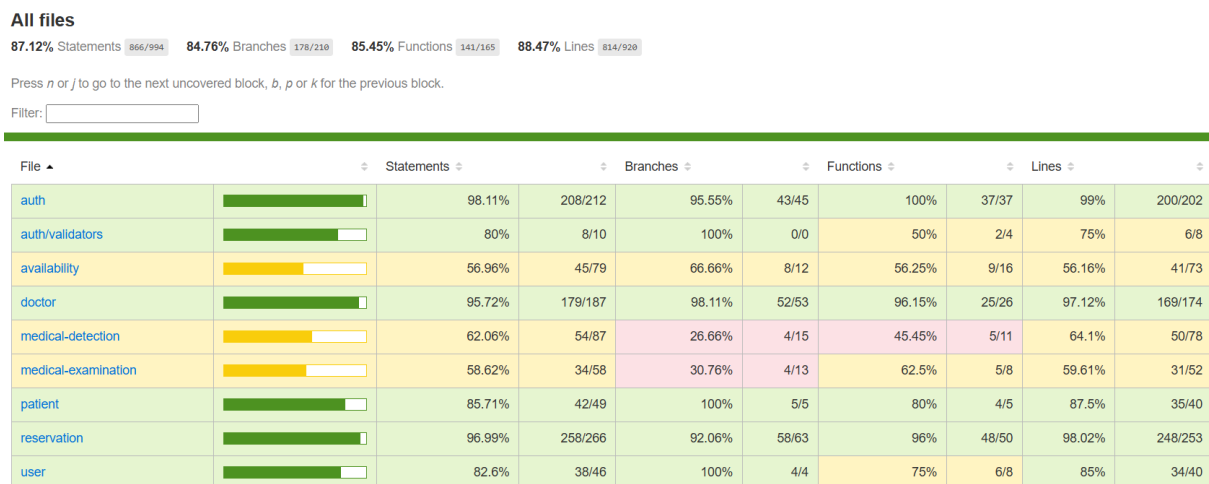


Figure 9.1: Copertura di test dopo la Change Request 3

Accanto agli unit test, è stato introdotto un test di integrazione/end-to-end specifico sul nuovo endpoint `POST /doctor/invite`

Questo test verifica il comportamento complessivo del sistema lungo l'intero flusso HTTP, utilizzando `supertest` e un'istanza `NestJS` configurata con repository mockati e dipendenze esterne emulate. In particolare, il test controlla:

- la creazione corretta del paziente e del relativo utente in caso di esecuzione corretta;
- l'impostazione del ruolo `PATIENT`;
- l'inizializzazione del flag `mustChangePassword` a `true`;
- il blocco della procedura in presenza di un paziente già esistente;
- la gestione del caso anomalo in cui il record del medico non sia presente;
- il corretto rifiuto di payload non validi da parte della `ValidationPipe`.

Il file `cr3-invite.e2e-spec.ts` consente quindi di verificare non solo la correttezza della logica di business, ma anche la coerenza dell'intero flusso dell'endpoint introdotto dalla Change Request.

9.3 Strategia di regression testing

Dopo l'implementazione della Change Request è stata rieseguita l'intera suite di test del progetto, con particolare attenzione ai moduli `Doctor`, `Reservation` e ai componenti precedentemente dipendenti dal flusso basato su `Invite`.

La regressione è stata necessaria in quanto la CR #3 non introduce soltanto una nuova funzionalità, ma rimuove completamente il precedente modulo `Invite` e semplifica diverse parti del codice che in precedenza contenevano logiche di fallback basate sugli inviti.

L'obiettivo della riesecuzione completa dei test è stato quindi verificare che:

- il nuovo onboarding sicuro del paziente fosse correttamente funzionante;
- la rimozione del modulo `Invite` non introducesse regressioni nei flussi già esistenti;
- i metodi rifattorizzati del `DoctorService` e del `ReservationService` rimanessero compatibili con il comportamento atteso.

9.4 Risultati dell'esecuzione dei test

L'esecuzione dei test ha confermato il corretto funzionamento della nuova logica introdotta dalla Change Request.

Gli unit test del modulo **Doctor** coprono in maniera completa sia il controller sia il service, validando i principali percorsi di esecuzione del nuovo onboarding e i casi di errore previsti. In particolare, il metodo `createInvite` è stato verificato sia nel caso positivo di creazione paziente+utente, sia nei casi di duplicato, conflitto, errore di hashing, paziente non trovato e fallimento nell'invio email.

Anche il test di integrazione sul nuovo endpoint `POST /doctor/invite` ha prodotto esito positivo sugli scenari previsti, dimostrando che:

- il sistema crea correttamente le entità **Patient** e **User**;
- il ruolo utente viene impostato correttamente;
- il flag `mustChangePassword` viene valorizzato come atteso;
- i vincoli di unicità vengono rispettati;
- la validazione del payload blocca richieste malformate prima dell'esecuzione del service.

Nel complesso, i test hanno mostrato che il nuovo flusso di onboarding è correttamente integrato nel sistema e che la rimozione del modello basato su inviti non ha prodotto malfunzionamenti nei percorsi verificati.

9.5 Coverage e miglioramento della qualità

L'attività di testing associata alla Change Request ha portato ad un incremento significativo della qualità del modulo **Doctor**, in quanto i componenti direttamente impattati dalla modifica risultano ora coperti integralmente dai test di unità.

La tabella seguente riporta il confronto tra la copertura prima e dopo l'introduzione della Change Request per il modulo impattato.

Metrica	Prima della CR	Dopo la CR
Statement Coverage	61%	96%
Branch Coverage	21%	98%
Line Coverage	63%	97%

Questo risultato è particolarmente rilevante poiché il service contiene la principale logica introdotta dalla CR #3, inclusi i controlli di unicità, la creazione delle entità, la generazione delle credenziali temporanee e la gestione del primo accesso sicuro.

L'aggiunta del test di integrazione consente inoltre di aumentare il livello di confidenza sul comportamento reale dell'endpoint, coprendo l'interazione tra controller, service, validazione e infrastruttura HTTP.

9.6 Valutazione finale del testing della CR

L'attività di testing svolta per la Change Request #3 ha consentito di validare in modo completo il nuovo flusso di onboarding sicuro dei pazienti.

La combinazione di unit test ad alta copertura e di un test di integrazione dedicato ha permesso di verificare sia la correttezza interna della logica applicativa sia il comportamento del nuovo endpoint nel suo contesto di esecuzione.

Dal punto di vista della manutenzione evolutiva, questa attività ha avuto un ruolo particolarmente importante, poiché la CR #3 ha comportato non solo l'introduzione di nuova logica, ma anche la rimozione di un intero modulo legacy e la semplificazione di diversi punti del sistema che dipendevano dall'entità **Invite**.

Nel complesso, i risultati ottenuti indicano che la funzionalità introdotta presenta un adeguato livello di affidabilità, integrazione e manutenibilità.